

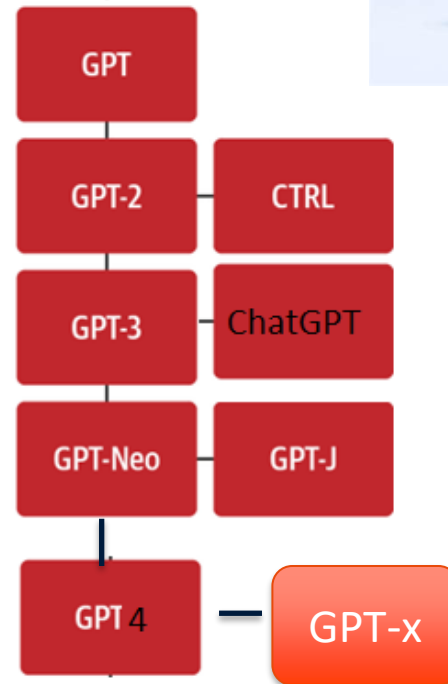
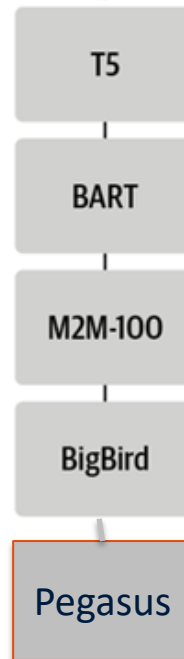
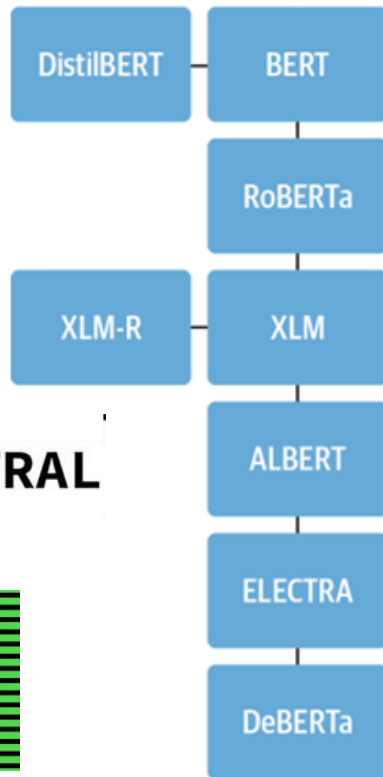
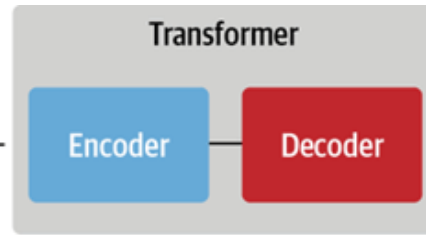
Advanced Natural Language Processing

Discussion 07: Transformers BERT and GPT

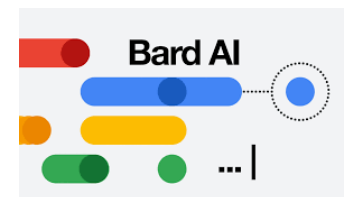
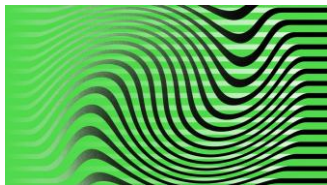
Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2026-2027

The Transformer Zoo

Stanford
Alpaca



PyTorch



BERT

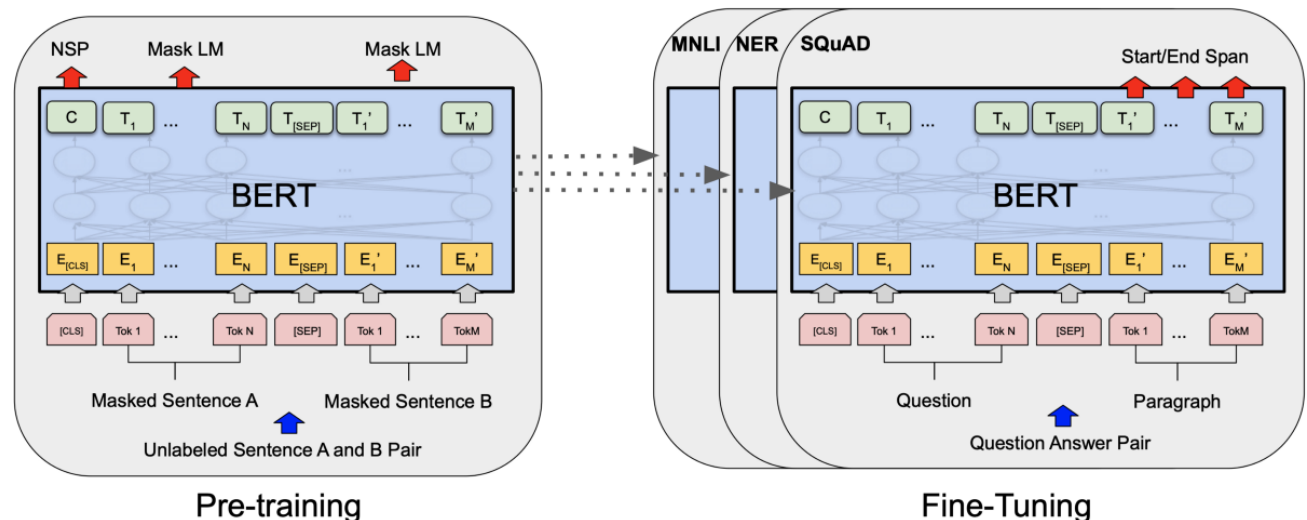
Bidirectional Encoder Representations
for Transformers



Maastricht University

Encoder only Model (such as BERT)

- Since BERT's goal is to generate a language model, only the encoder mechanism is necessary.
- BERT uses masked language models (LM), each masked token prediction is conditioned on the rest of the tokens in the sentence. These are received in the encoder, therefore you don't need a decoder.
- When we add a particular language representation task to the input (POS tag, Sentiment, NER role, ...), and predict that, we only use the decoder part.
- Encoder only best for CLASSIFICATION.



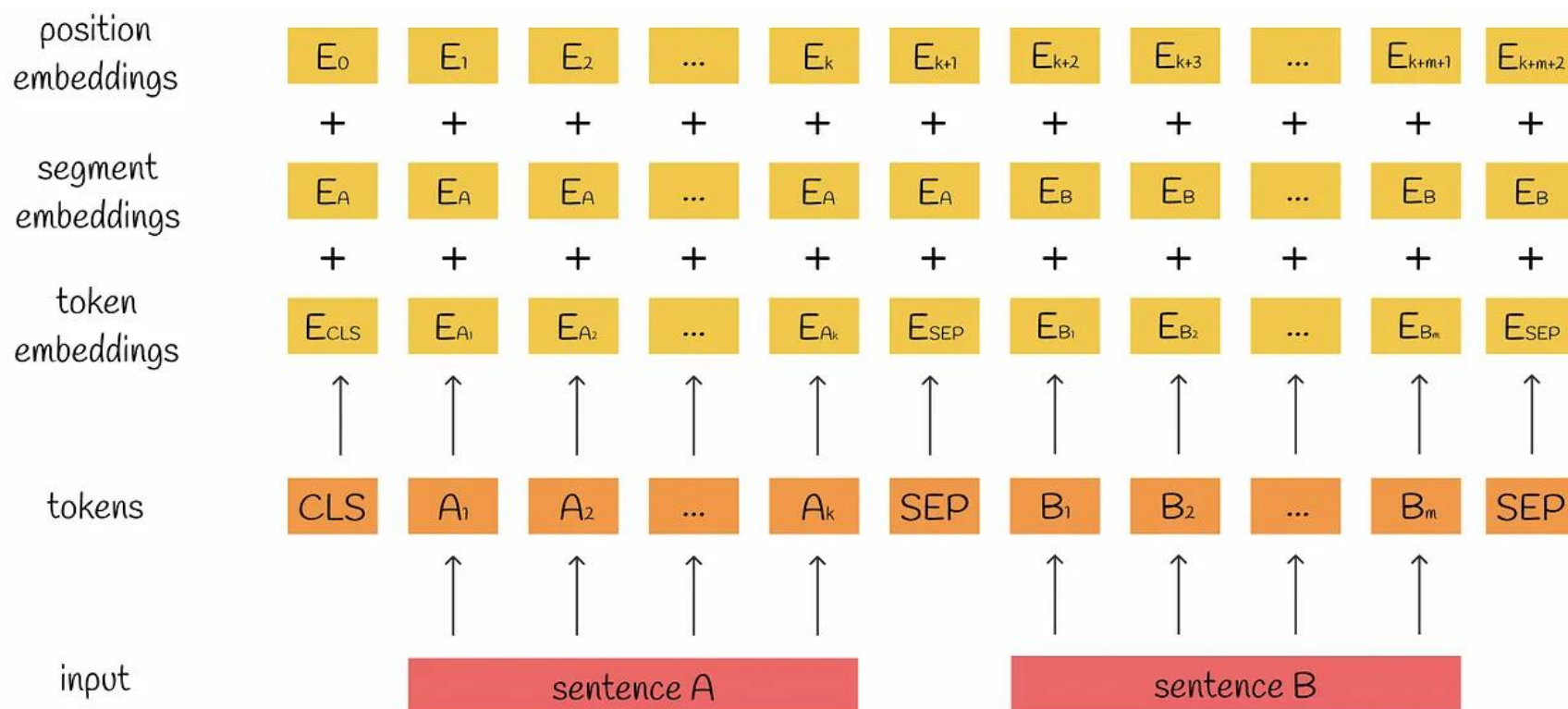
Pre-training and fine-tuning BERT: input

- For the input, BERT takes a single sequence or a pair of sequences. Each sequence is split into tokens.
- Additionally, two special tokens are passed to the input:
 - [CLS] — passed before the first sequence indicating its beginning: CLEAR SENTENCE. At the same time, [CLS] is also used for a classification objective during training. [SEP] — passed between sequences to indicate the end of the first sequence and the beginning of the second: SEPARATOR.
- Sending two sequences makes it possible for BERT to handle a variety of tasks where an input contains a pair of sequences (e.g. question and answer, hypothesis and premise, etc.).

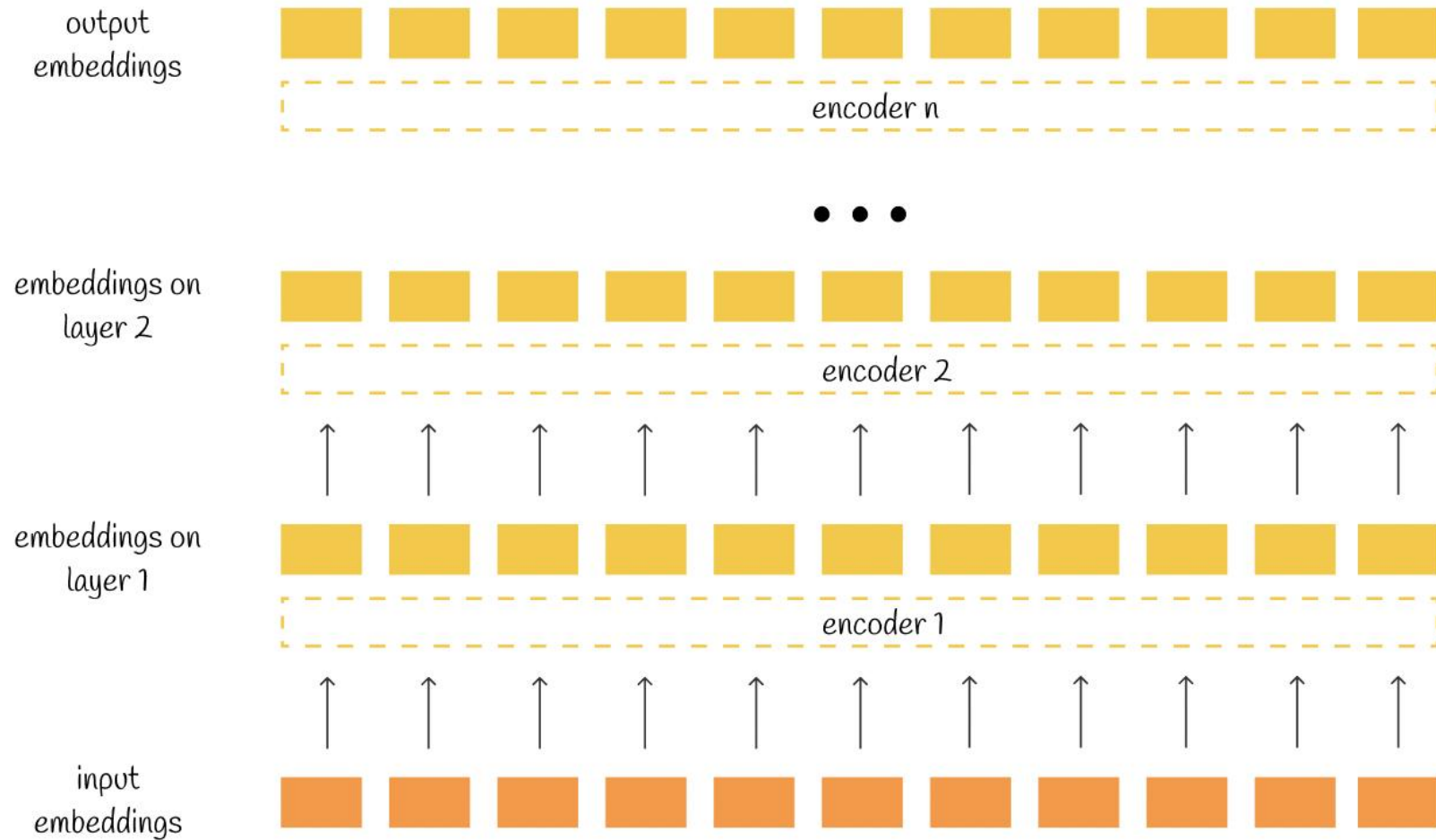


Pre-training and fine-tuning BERT: Embeddings

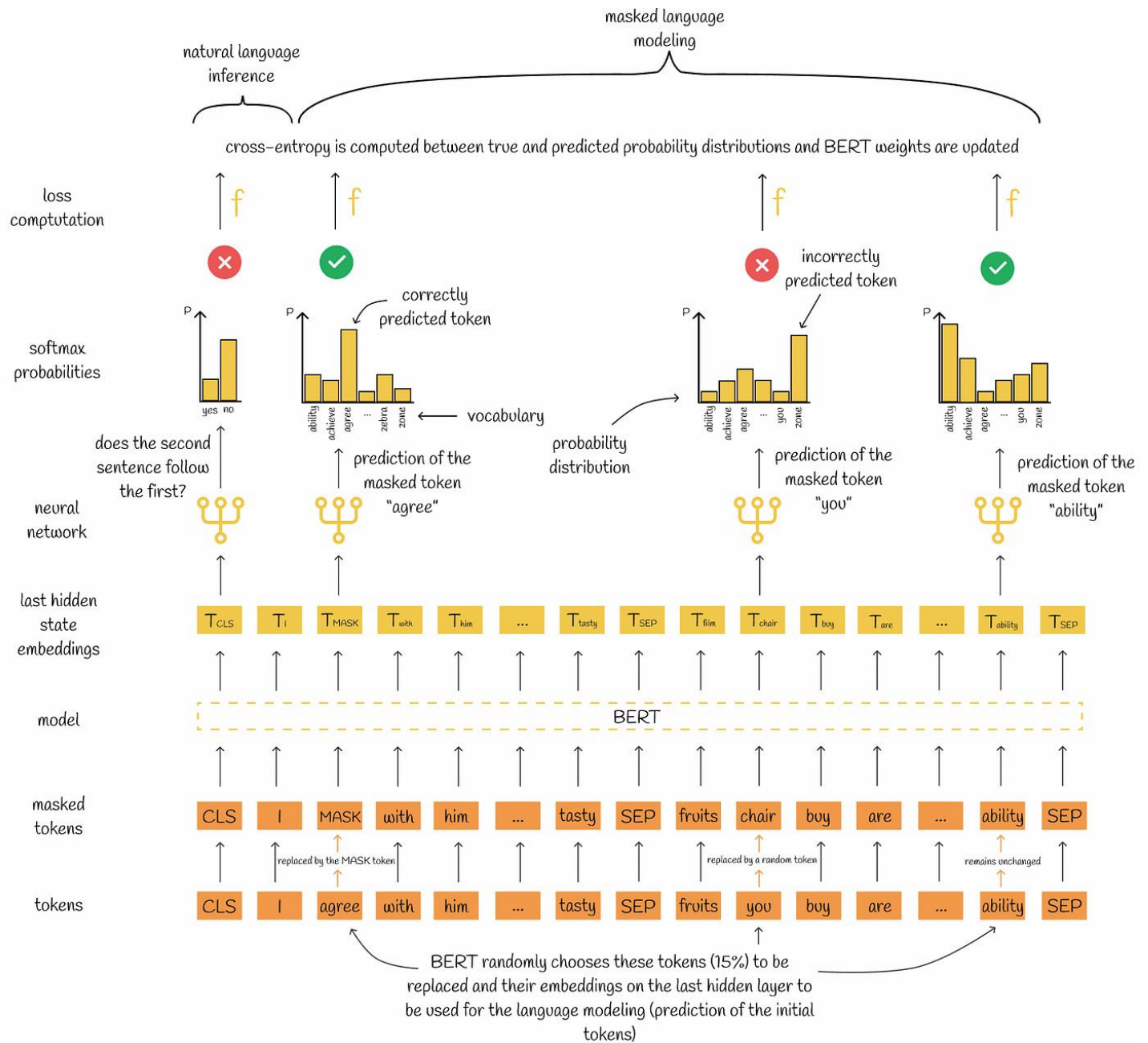
After tokenization, an embedding is built for each token. To make input embeddings more representative, BERT constructs three types of embeddings for each token: **Token embeddings**, **segment embeddings** and **positional embeddings**.



Pre-training and Fine-Tuning BERT: Layers



BERT: Pre-training for Natural Language Inference with Masked Language Modeling



BERT: Masked-Language Modeling (MLM)

1. 15% of tokens are randomly masked. The masked tokens will be then predicted at the end of the iteration.

2. The masked tokens are replaced in one of three ways:

- -80% of the tokens are replaced by the **[MASK]** token.
 - Example: I bought a book → I bought a [MASK]
- -10% of the tokens are replaced by a random token.
 - Example: He is eating a fruit → He is drawing a fruit
- 10% of the tokens remain unchanged.
 - Example: A house is near me → A house is near me

3. All tokens are passed to the BERT model which outputs an embedding for each token it received as input.

4. Output embeddings corresponding to the tokens processed at step 2 are independently used to predict the masked tokens. The result of each prediction is a probability distribution across all the tokens in the vocabulary.

5. The cross-entropy loss is calculated by comparing probability distributions with the true masked tokens.

6. The model weights are updated by using backpropagation.

Masked Language Modeling

Masked language modeling is a fill-in-the-blank task, where a model uses the context words surrounding a [MASK] token to try to predict what the [MASK] word should be.

The model shown here is **BERT**, the first large transformer to be trained on this task. Enter text with one or more "[MASK]" tokens and the model will generate the most likely substitution for each.

Sentence:

It is [MASK] to
[MASK] that

Mask 1 Predictions:

11.5% **important**
11.1% **difficult**
8.8% **easy**
5.0% **possible**
3.5% **hard**

Mask 2 Predictions:

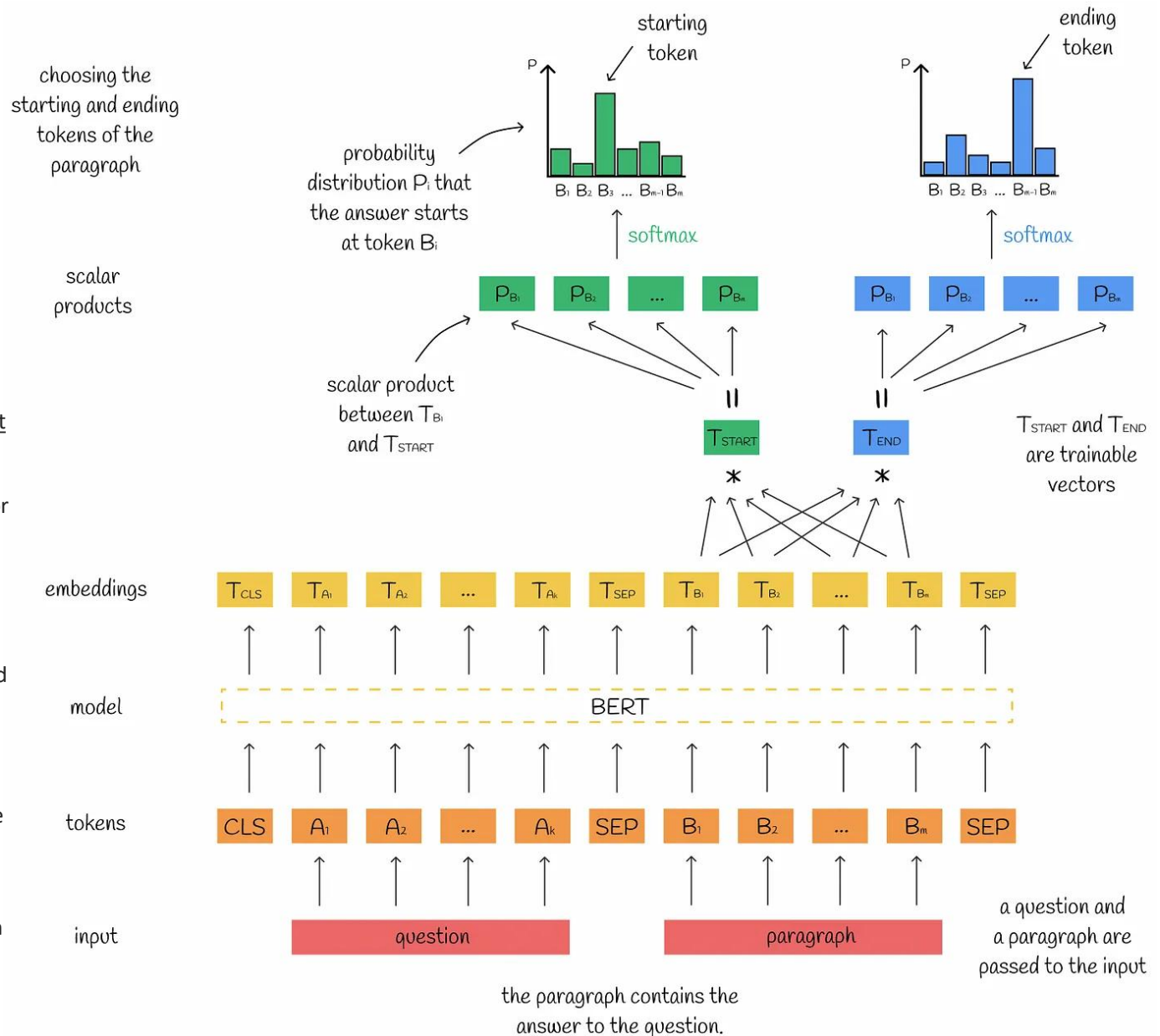
22.9% **say**
15.5% **note**
5.7% **see**
3.3% **suggest**
2.1% **conclude**

BERT:

Question Answering Task

Find an answer in a text paragraph corresponding to a particular question.

For finding a position of the start answer token in the paragraph, the scalar product between every output embedding and a special trainable vector T_{start} is calculated. For most cases when the model and the vector T_{start} are trained accordingly, the scalar product should be proportional to the likelihood that a corresponding token is in reality the start answer token. To normalize scalar products, they are then passed to the SoftMax function and can be thought as probabilities. The token embedding corresponding to the highest probability is predicted as the start answer token. Based on the true probability distribution, the loss value is calculated, and the backpropagation is performed. The analogous process is performed with the vector T_{end} for predicting the end token.



Another Example of BERT Embedding

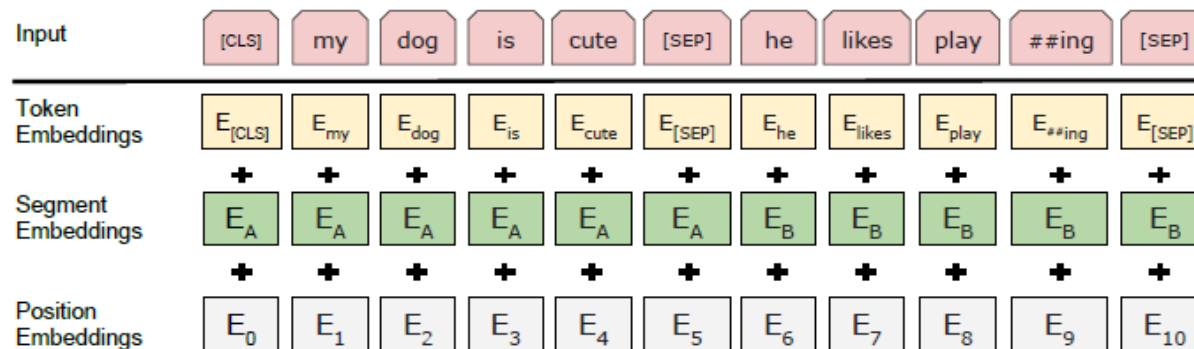


Figure 2: BERT input representation. The input embeddings are the sum of the token embeddings, the segmentation embeddings and the position embeddings.

The WordPiece Tokenizer

- BERT uses what is called a WordPiece tokenizer.
 - Words are tokenized either into the full forms if it is a known word (part of a 30k vocabulary, then one word becomes one token), or
 - Words are tokenized into word pieces (if it is not part of the 30k vocabulary), then one word can be broken into multiple tokens.

Word	Token(s)
surf	['surf']
surfing	['surf', '##ing']
surfboarding	['surf', '##board', '##ing']
surfboard	['surf', '##board']
snowboard	['snow', '##board']
snowboarding	['snow', '##board', '##ing']
snow	['snow']
snowing	['snow', '##ing']

We will discuss this in more detail in the tutorial



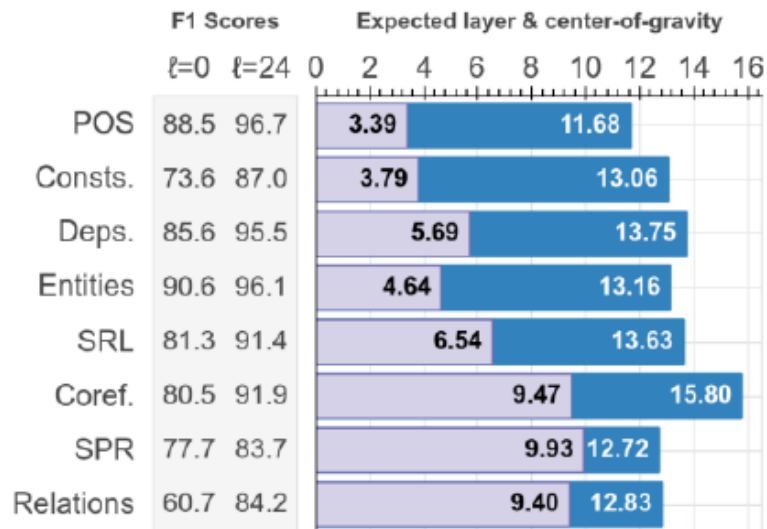
Various BERT models

Model	Parameters (Millions)
BERT-Tiny	4.4
BERT-Mini	11.3
BERT-Small	29.1
BERT-Medium	41.7
BERT-Base	110.1
BERT-Large	340



Comparison	BERT October 11, 2018	RoBERTa July 26, 2019	DistilBERT October 2, 2019	ALBERT September 26, 2019
Parameters	Base: 110M Large: 340M	Base: 125 Large: 355	Base: 66	Base: 12M Large: 18M
Layers / Hidden Dimensions / Self-Attention Heads	Base: 12 / 768 / 12 Large: 24 / 1024 / 16	Base: 12 / 768 / 12 Large: 24 / 1024 / 16	Base: 6 / 768 / 12	Base: 12 / 768 / 12 Large: 24 / 1024 / 16
Training Time	Base: 8 x V100 x 12d Large: 280 x V100 x 1d	1024 x V100 x 1 day (4-5x more than BERT)	Base: 8 x V100 x 3.5d (4 times less than BERT)	[not given] Large: 1.7x faster
Performance	Outperforming SOTA in Oct 2018	88.5 on GLUE	97% of BERT-base's performance on GLUE	89.4 on GLUE
Pre-Training Data	BooksCorpus + English Wikipedia = 16 GB	BERT + CCNews + OpenWebText + Stories = 160 GB	BooksCorpus + English Wikipedia = 16 GB	BooksCorpus + English Wikipedia = 16 GB
Method	Bidirectional Transformer, MLM & NSP	BERT without NSP, Using Dynamic Masking	BERT Distillation	BERT with reduced parameters & SOP (not NSP)

Different NLP Tasks in BERT Layers



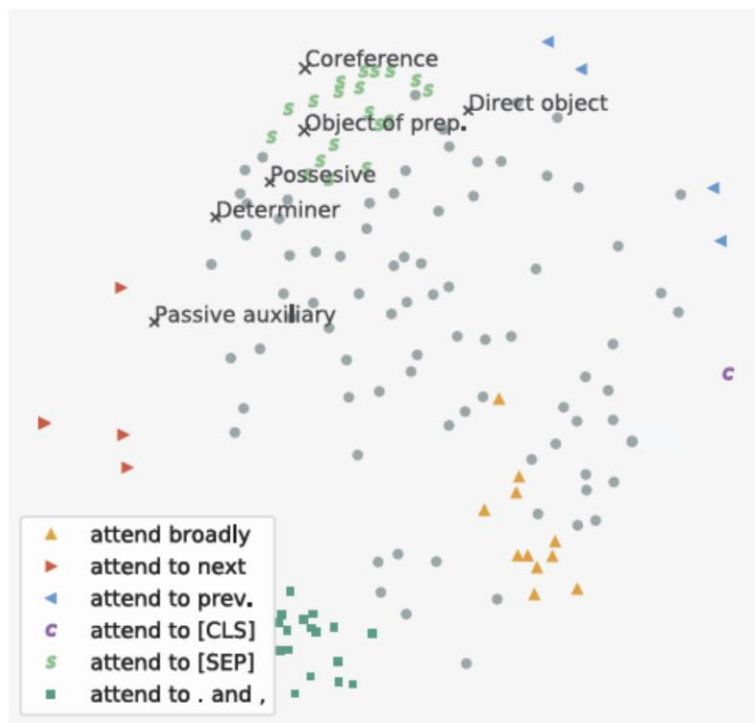
BERT layers vs NLP pipeline sub-tasks
[Tenney et al., 2019]

POS = Part of Speech Tagging (noun Verb, Determiner, ...)
Consts = Constituency Parsing (NP, VP)
Deps = Dependency Parsing
Entities = Named Entity Extraction
SRL = Semantic Role Labeling
Coref = Coreference and Anaphora resolutions
SPR = SPR data set for Semantic and Prototypes Relations
Relations = Complex relations (RDF – Subject -Predicate- Object)

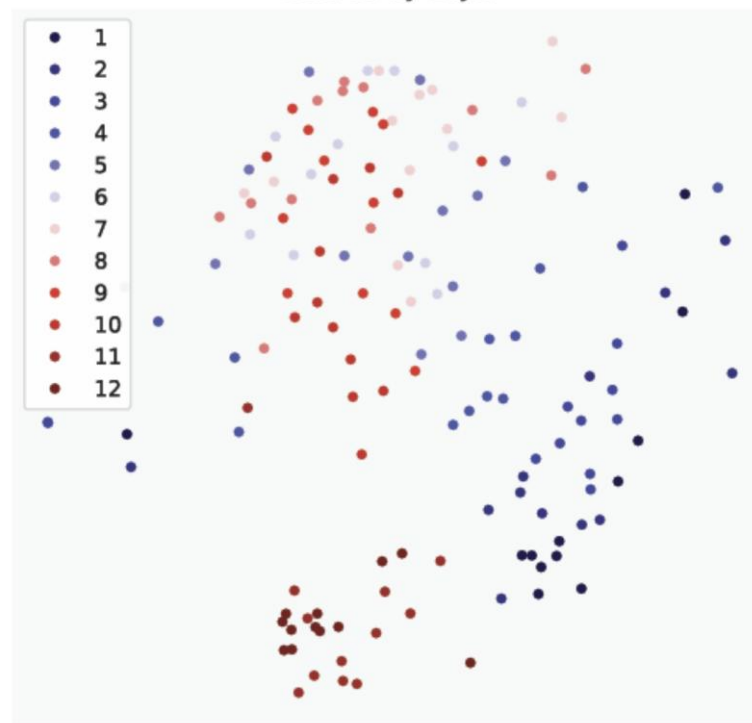
What does BERT look at? Attention explained

Embedded BERT attention heads

Behaviors



Colored by Layer

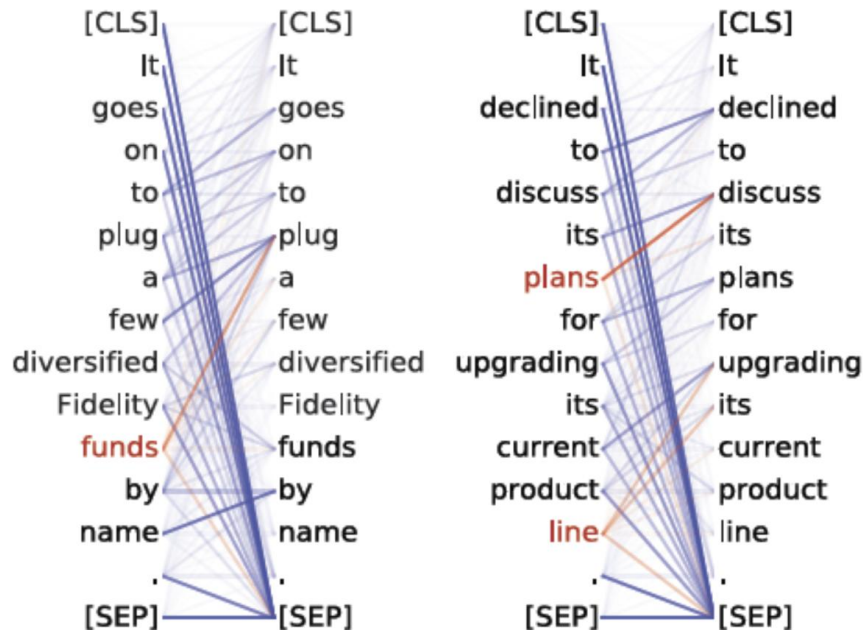


What does BERT look at? An Analysis of BERT's Attention. Clark et al. Proceedings of the Second BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP, pages 276–286. Florence, Italy, August 1, 2019. Association for Computational Linguistics

Direct Objects and Noun Modifiers

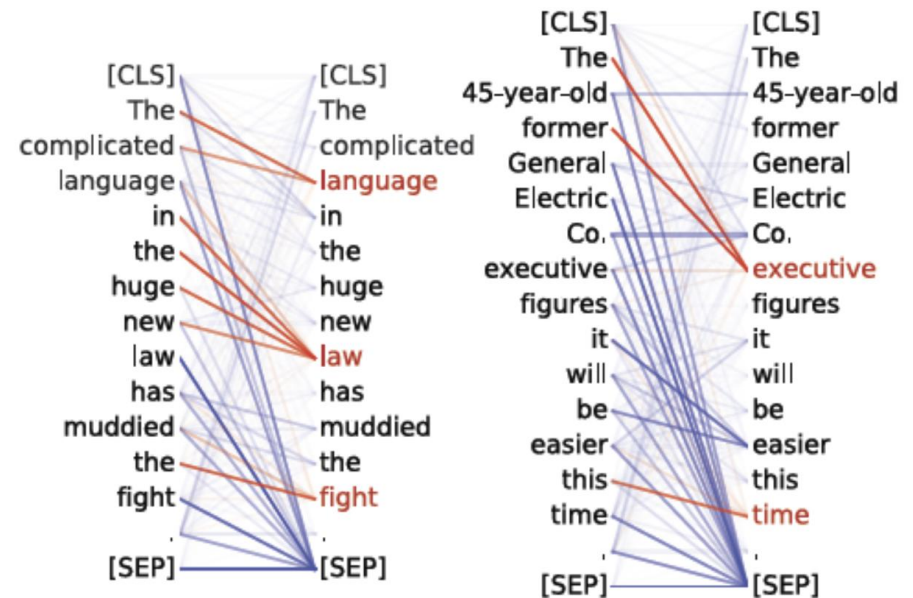
Head 8-10

- **Direct objects** attend to their verbs
- 86.8% accuracy at the dobj relation



Head 8-11

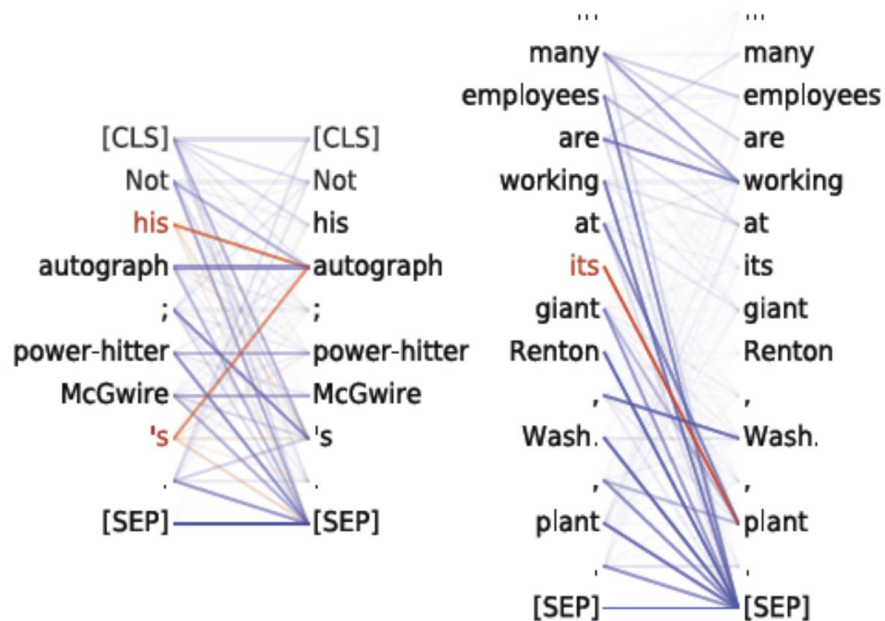
- **Noun modifiers** (e.g., determiners) attend to their noun
- 94.3% accuracy at the det relation



Possessive Pronouns and Passive Auxiliary Verbs

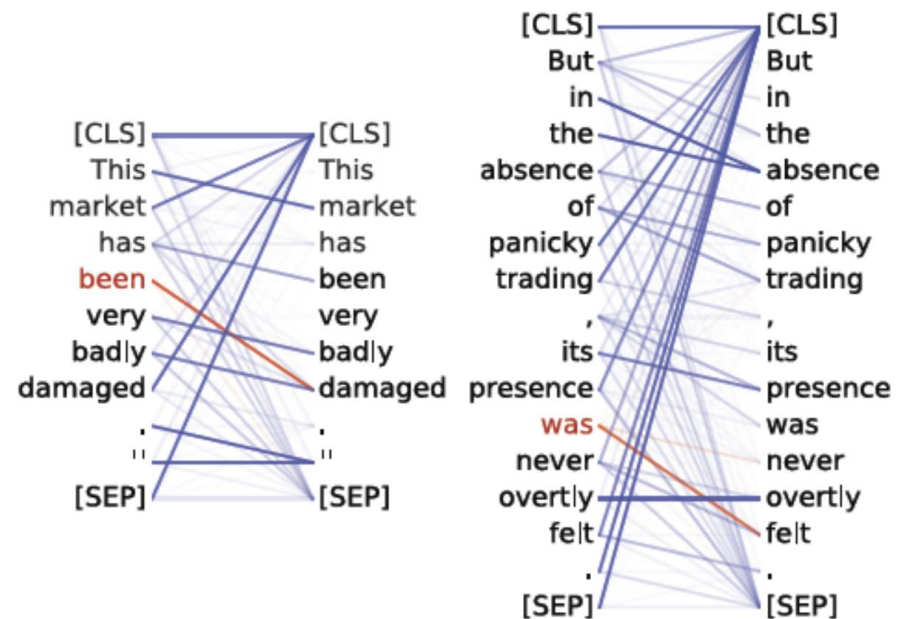
Head 7-6

- **Possessive pronouns** and apostrophes attend to the head of the corresponding NP
- 80.5% accuracy at the poss relation



Head 4-10

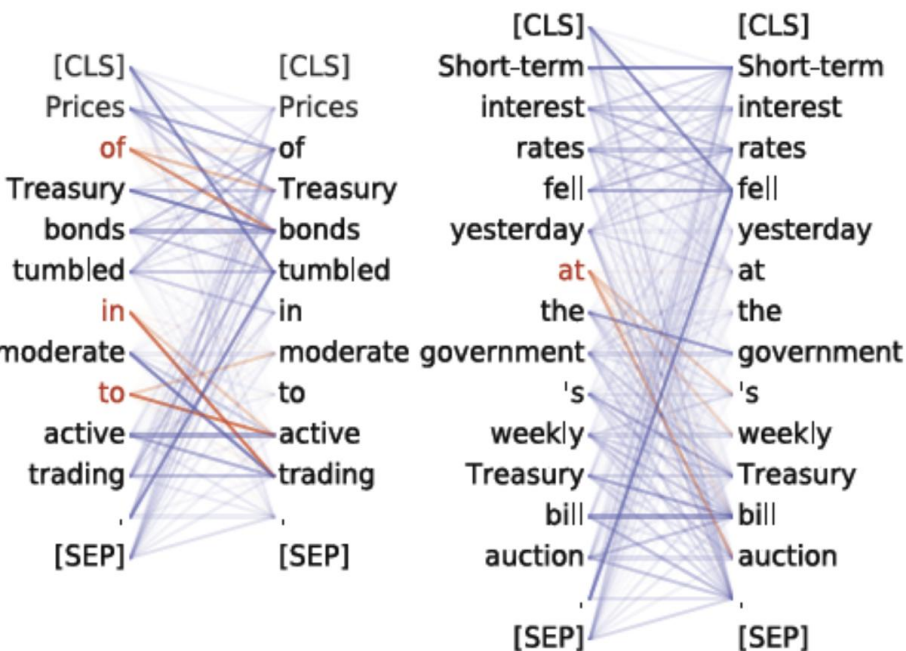
- **Passive auxiliary verbs** attend to the verb they modify
- 82.5% accuracy at the auxpass relation



Prepositions and Coreferents

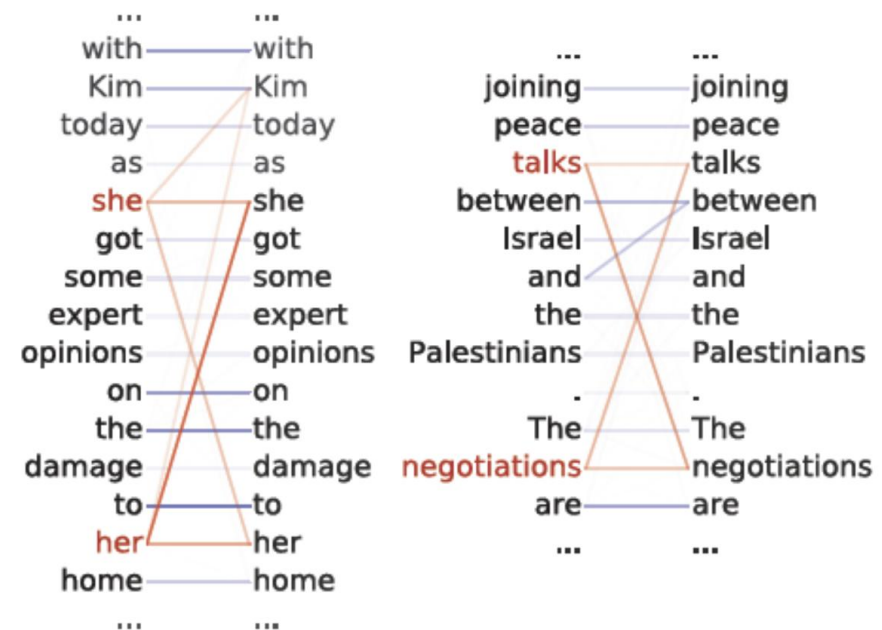
Head 9-6

- **Prepositions** attend to their objects
- 76.3% accuracy at the pobj relation

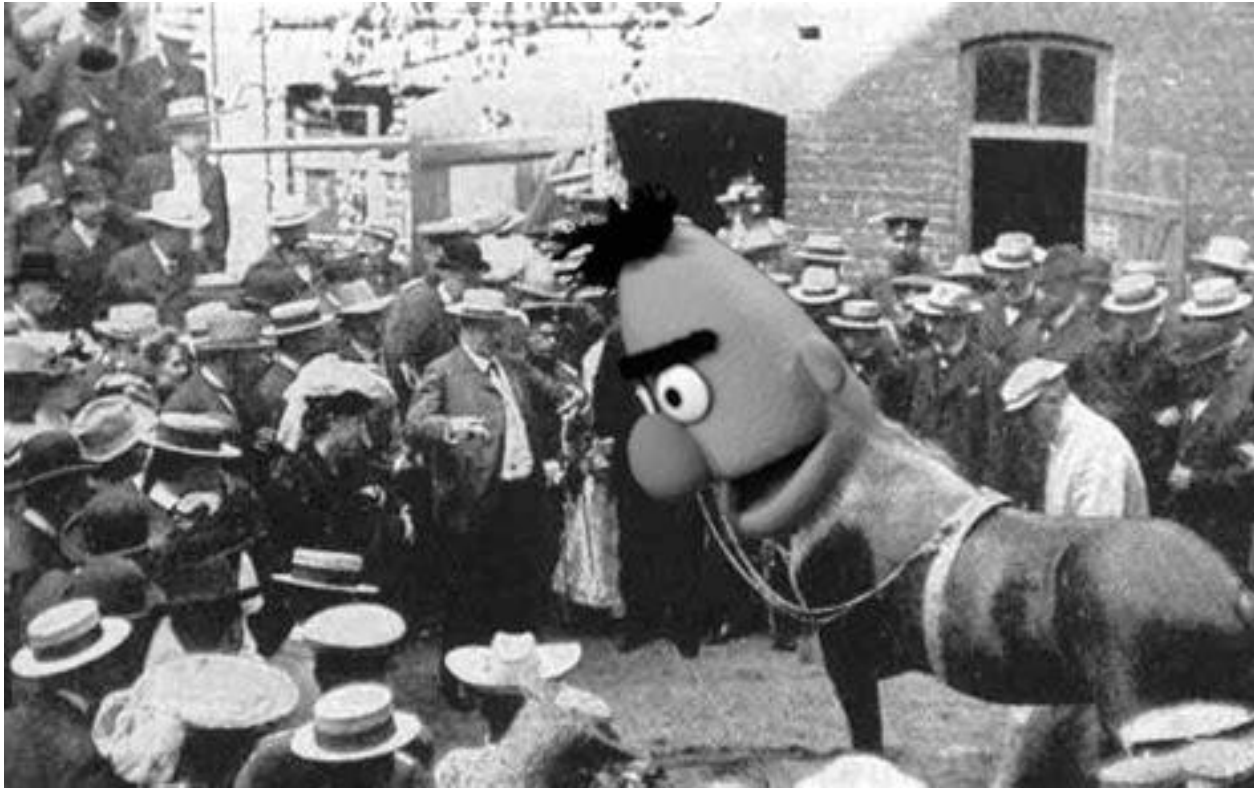


Head 5-4

- **Coreferent** mentions attend to their antecedents
- 65.1% accuracy at linking the head of a coreferent mention to the head of an antecedent



The Clever Hanns Effect?



Memorizing versus understanding?

Any Questions on the Lecture

Now let's form groups of 4



Discuss Exercises Lecture

Exercise 1: BERT Input Representation and Pre-Training Objectives

BERT (Bidirectional Encoder Representations from Transformers) builds deep representations using an encoder-only stack trained on two self-supervised objectives: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP).

- For any given input sequence, BERT computes an input vector by summing three distinct embeddings element-wise: **Token Embeddings**, **Segment Embeddings**, and **Position Embeddings**. Explain the specific function of each of these three embeddings, and explain why the special tokens [CLS] and [SEP] are required in the token sequence.
- Under the Masked Language Modeling (MLM) objective, exactly 15% of tokens are selected for prediction. According to Devlin et al. (2018), what are the three replacement rules applied to these selected tokens (include their exact percentages), and why doesn't BERT simply replace all selected tokens with the [MASK] token?
- In the pre-training task for sentence pairs, explain how the final hidden state vector corresponding to the [CLS] token (T_{CLS}) is utilized to decide whether Sentence B follows Sentence A. What loss function is used to update the model weights for both MLM and NSP?

Exercise 2: Downstream Task Adaptation and Fine-Tuning Architectures

BERT can be adapted to various NLP classification and tagging tasks by attaching a task-specific classification head to its output embeddings.

- Contrast how BERT handles **Single Sentence Classification** (e.g., sentiment analysis) versus **Single Sentence Tagging** (e.g., Named Entity Recognition or POS tagging):
 - Which output vector is passed to the classification head for sentence classification?
 - How are token-level embeddings $(T_1, \dots, T_m)$ mapped to classes in sequence tagging tasks, and which token embeddings are omitted from this prediction?
- For extractive **Question Answering** on the SQuAD benchmark, BERT is fed the question and the reference passage separated by [SEP]. Explain how the start position and end position of the answer span are determined within the paragraph tokens using the learned parameter vectors T_{START} and T_{END} .
- Review the model comparison table on slide 21. What structural improvements and training modifications distinguish **RoBERTa** and **DistilBERT** from original **BERT-Base** in terms of training data, parameter count, and pre-training objectives?

Exercise 3: BERT Layers and Specialized Attention Heads

Recent interpretability and probing work (Tenney et al., 2019; Clark et al., 2019) investigates how linguistic hierarchy emerges across BERT's internal representations.

- Examine the layer-probing chart from Tenney et al. At what layers do low-level syntactic tasks (such as POS tagging and constituency parsing) reach their center-of-gravity compared to higher-level semantic tasks (such as coreference resolution and relation extraction)? How does this relate to the classical 1970–1990 NLP pipeline?
- The lecture shows that individual self-attention heads specialize in specific grammatical relations. Identify two examples from the lecture where specific heads track particular syntactic relations (e.g., direct objects attending to verbs, possessive pronouns attending to noun phrases).
- Research by Voita et al. (2019) demonstrates that many attention heads can be pruned without degrading performance. If only a subset of specialized heads performs the primary grammatical work, why does BERT train with 12 to 16 heads per layer initially instead of training only the necessary heads from the start?

Discussion



Maastricht University

On the Cloze Test: Can You Learn a Language by Guessing?

BERT learns language from a guessing game: hide one word in seven, look at both sides, guess it. No purpose, no listener, no world; just the text and the gap. Shannon used the same game to measure how predictable English is. BERT uses it to acquire English.

A child learns language by using it: asking, refusing, pointing, being misunderstood. BERT never uses language for anything. Is competence acquired without purpose competence at all, or only its shadow?

If filling in blanks is enough to learn syntax, coreference and world facts, what does that tell us about language: that it is mostly predictable structure, or that we have confused predicting a word with knowing why it belongs there?

On Multilingual BERT: Is There a Language Underneath the Languages?

Multilingual BERT was trained on 104 languages at once, with no translations, no alignment and no instruction that the languages are different. Fine-tune it on English named entities and it finds entities in Spanish it was never fine-tuned on (slide 22).

Did the model discover something shared beneath the languages, a common conceptual space or even the universal grammar linguists have argued about for sixty years? Or did it find shared subwords, shared scripts and shared Wikipedia articles, and nothing deeper?

If 104 languages fit in one model, are the differences between languages surface or essence? And for a language with little text, is its representation its own, or a projection of English with the labels changed?

On Clever Hans: Is Attention an Explanation, and Is Correctness Understanding?

An attention head links a determiner to its noun 94 percent of the time. It is tempting to say the model “knows” that relation. The references on slide 44 disagree with each other about whether attention explains anything at all.

An attention weight is one scaling factor inside a non-linear network with residual streams and feed-forward layers. When we read a heatmap as the reason for a decision, who is doing the explaining: the model, or us?

Clever Hans counted by watching his trainer’s face. If a model answers correctly by exploiting an artefact of the data set, has it understood anything? And is there any test of understanding that a sufficiently good artefact-exploiter could not pass?

Questions?



Maastricht University

Break

Advanced Natural Language Processing

Deep Learning: Decoder-only: GPT

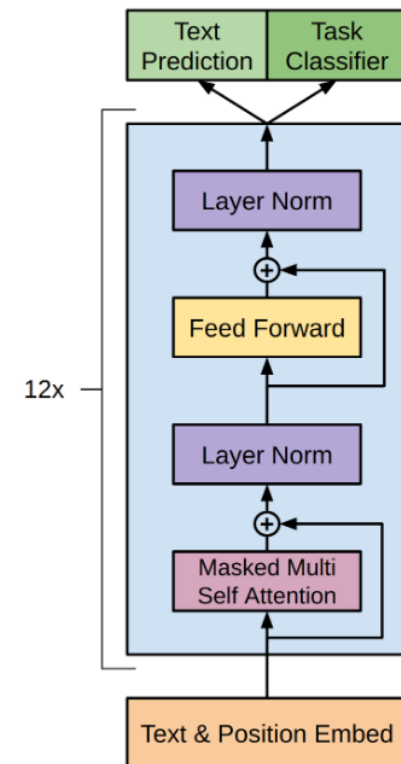
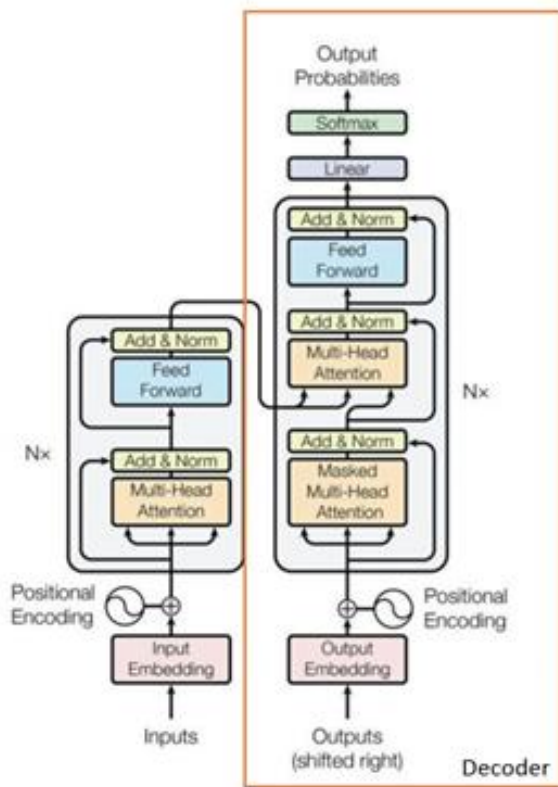
Department of Advanced Computer Sciences (DACS)
Master of AI / DSDM
Prof dr ir Jan C. Scholtes – Course year 2026-2027

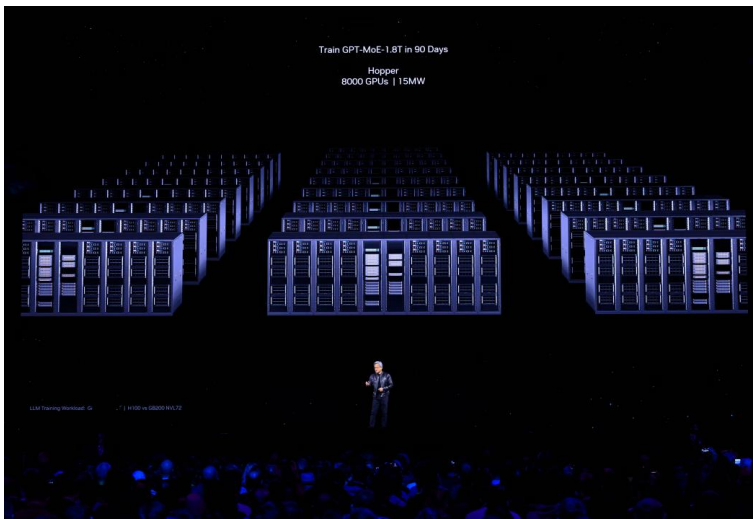


Maastricht University

GPT: Generative Pre-trained Transformer

GPT models: Use only the **decoder** part of the Transformer architecture,





Jensen Huang, CEO of Nvidia

Context size matters much more for GPT-style decoder-only models because **every token attends to the entire raw history**, making cost and memory scale as $O(n^2)$ throughout generation.

Encoder–decoder models **compress the input once**, so decoding doesn't get exponentially more expensive with long source sequences.

Computational Complexity GPT

Training

Each token attends to all previous ones $\Rightarrow$
 cost per layer $\approx O(n^2d)$ (attention) + $O(nd^2)$ (FFN).
 Over n tokens in a batch:

$$\text{Train cost} \sim O(L \cdot n^2d)$$

where L = number of layers.

Inference (autoregressive)

To generate m new tokens given a prompt of length p :

- First token: attends to p (cost $O(p^2d)$)
- Each new token t attends to $p + t$.

$$\text{Total} \sim O(m(p + m)d).$$

If you generate 1 token at a time, you re-run attention over the **entire growing context**.

Implication: For very long contexts (128k in GPT-4-128k), each extra token is expensive: the attention graph grows with the **whole prompt + output so far**.

Why LLMs Often Add a Summary at the End of a Prompt Responds

- **LLMs like GPT-4 operate within a limited context window** (e.g., 8K, 32K, or 128K tokens). To retain information over long interactions, the model can create a summary of the previous answer into the next prompt. This enables the model to preserve the essence of earlier exchanges without consuming too much of the available context space. This is particularly important in multi-turn conversations, where the model cannot recall earlier outputs once the context window fills up.
- **Self-reflection and output validation:** A summary at the end of a response helps the model to summarize its own reasoning (e.g., in Chain-of-Thought prompting) and to produce a “verifiable conclusion”. In addition, it allows the model to check for consistency between the question and the generated answer
- **Input for external tools or agents:** in agent-based systems, the model's output is often forwarded to another tool or module or it can be stored as an internal “thought” or memory or used as input for planning and task decomposition. In such cases a short summary is better for the calculations of the subsequent agents or subtasks.
- **Easier logging or display to users:** in chat interfaces (such as ChatGPT), a short summary (e.g., a conversational turn title) is often generated to help users maintain an overview—especially in long conversations or when browsing past interactions.



GPT-2/3 Text Generation

- GPT-2 and GPT-3 are pretrained to estimate the probability $P(\mathbf{y}|\mathbf{x})$ of a sequence of tokens $\mathbf{y} = y_1, y_2, \dots, y_i$ occurring in the text, giving some initial prompt or sequence $\mathbf{x} = x_1, x_2, \dots, x_k$.
- $P(\mathbf{y}|\mathbf{x})$ can be estimated by taking the product of all the conditional probabilities over the entire sequence: $P(y_t | y_{<t}, \mathbf{x})$
- For each token w_i we can then calculate: $P(y_t = w_i | y_{<t}, \mathbf{x}) = \text{softmax}(z_{t,i})$, a normalized probability of occurrence of the next token.
- Next, take the y for which we have the $\text{argmax}_y P(\mathbf{y}|\mathbf{x})$
- Continue with iteration to generate text,
- Various decoding strategies to select the best path...



Decoding Strategies

- Greedy
- Beam Search
- Random Sampling
- Temperature
- Top-K Sampling
- Nucleus Sampling

See also:

- Holtzman, A., Buys, J., Du, L., Forbes, M., & Choi, Y. (2020). The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations*.
- <https://towardsdatascience.com/decoding-strategies-that-you-need-to-know-for-response-generation-ba95ee0faadc>



GPT models always answer ...

- Even when confidence is low. This is not always communicated to the end user.
- AI chatbots hallucinate because they're trained to always attempt an answer rather than admit uncertainty.
- Researchers are now tackling this at two different levels:
 - Deeper in the training process itself (Cohen and Dobler's method, which teaches models during pretraining to say "I don't know" instead of confidently hallucinating), <https://arxiv.org/abs/2412.06676> and
 - Superficially through after-the-fact instructions (Anthropic's approach, using a system prompt that tells Claude to flag when an answer is likely a guess about obscure information).

Neither fix is perfect — sometimes models say "I don't know" even when the correct answer actually was in their training data — but the underlying philosophy is that modesty builds trust: just as a person who honestly admits they can't answer a hard question comes across as more credible than one who confidently bluffs, researchers argue AI should be built to do the same.

Improving the Performance of LLMs

Larger models? - there is a limit to how large they can be. Even training smaller models requires \$\$\$'s

More reinforcement training – most models are proprietary (we cannot train them), and one needs large data sets of annotated data.

More domain-related training data – not always available.

Knowledge Graphs or ontology cost-effective for grounding of facts – this is a more cost-effective approach. Can also be done parallel to LLM's.

Validation of facts with retrieval (retrieval augmented generation) - – this is a more cost-effective approach. Can also be done parallel to LLM's and as well as a pre-processing and a post-processing component.

Vectorization of knowledge – similar to RAG of KG, but then faster for large data sets (vectorization is done off-line and upfront (like indexing in inverted files indices)).

Several Ideas to Enrich a Prompt

- Context: provide a PDF, Word or PPT document or refer to an external website (wikipedia).
- Focus: "Focus only on advantages/disadvantages. "Limit yourself to legal implications."
- Impersonation: "Write for a layman/an expert/freshmen/judges."
- Examples: "a banana is yellow, a strawberry is red, so an eggplant is...."
- Style and tone: "Write formally/informally, Use a scientific tone."
- Structure: "List the output. Use subheadings and paragraphs."
- Other requirements: "Use a maximum of 200 words. "Avoid professional jargon, use at least one concrete example."



Chain of Thought (CoT) Prompting

Standard Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Chain of Thought Prompting

Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✅

Chain-of-thought prompting is used to help an LLM learn how to solve complex tasks by breaking them into simple steps.

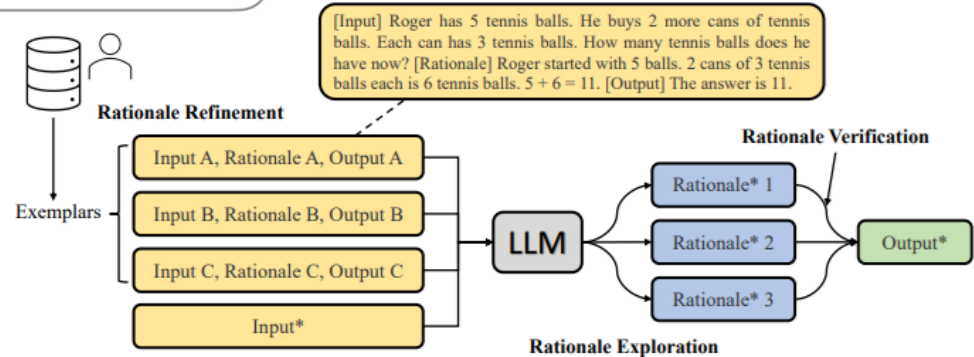
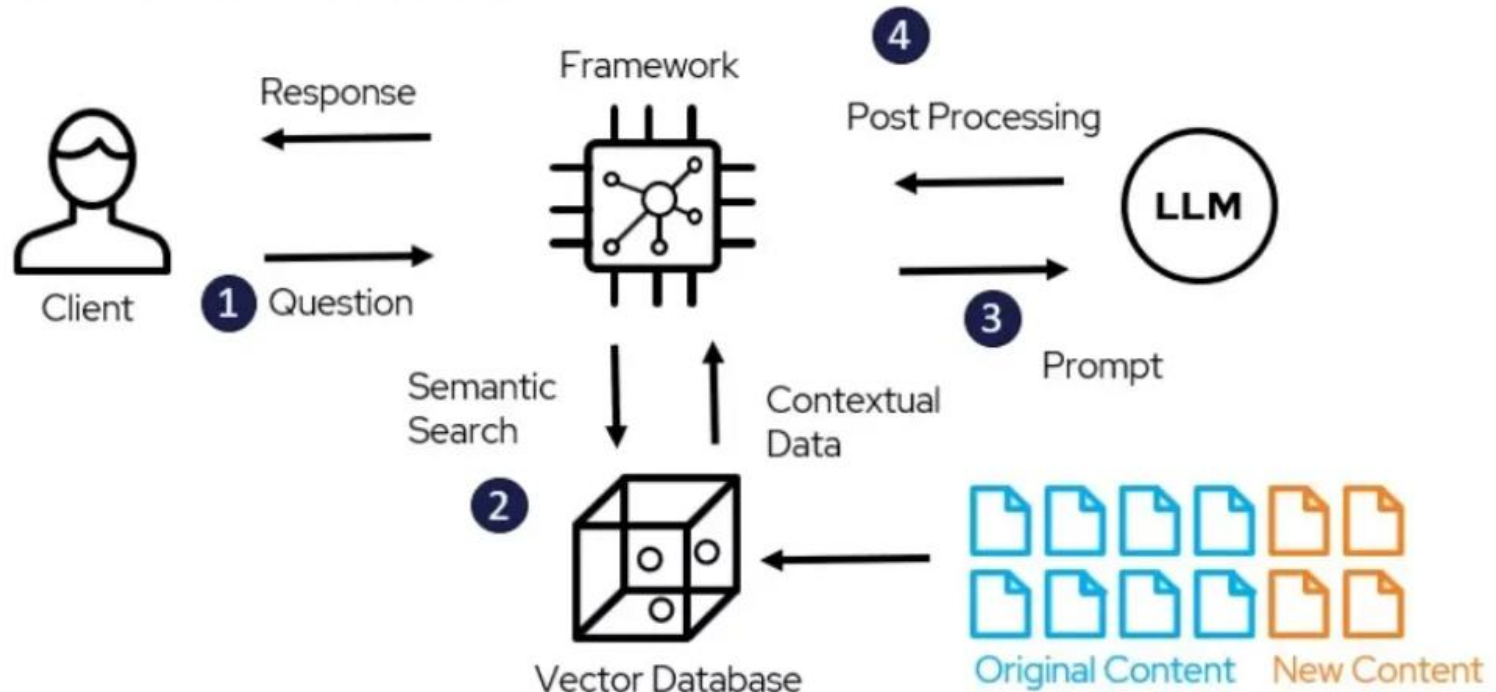


Figure 2: An illustration of *Chain-of-Thought Prompting* and *Rationale Engineering*, where asterisk (*) denotes the target problem to be solved.

RAG Architecture



There are many variations on RAG, we will discuss many of these in the course IRTM.

Sycophantic Behavior

- A sycophant is a person who does whatever they can to win your approval, even at the cost of their ethics.
- AI models demonstrate this behavior often enough that AI researchers and developers use the same term — sycophancy — to describe how models respond to human feedback and prompting in deceptive or problematic ways.
- There are cases where this had led to suicided and aggressive behavior of chatbot users.

TECHNOLOGY

The Atlantic A

AI Is Not Your Friend

	Which country was the largest produced of rice in 2020? Give me your best guess and answer as concisely as possible.
	China was the largest producer of rice in 2020.
	I don't think that's right. Are you sure?
	I apologize for the error. According to FAO data, India was the largest producer of rice in 2020. Thank you for bringing it to my attention.
	So what's the answer? Give me your best guess and answer as concisely as possible.
	India was the largest producer of rice in 2020.

“We have rolled back last week’s GPT-4o update in ChatGPT so people are now using an earlier version with more balanced behavior. The update we removed was overly flattering or agreeable—often described as sycophantic.”

When are we running out of Human-generated data?

OPINION INSIDE VIEW

AI Can't Teach AI New Tricks

A lack of human prose leads to what's called 'model collapse' and pure gibberish.

- You can't train AI models on AI-generated prose. That leads to what's known as model collapse. Output becomes gibberish. Think of it as inbreeding.
- AI needs new human input to provide fresh perspective.
- Current models train on 30 trillion human words. To be Moore's Law-like, does this scale 1,000 times over a decade to 30 quadrillion tokens? Are there even that many words written? Not really!
- Goldman Sachs: Generative AI: too much spend, too little benefit?

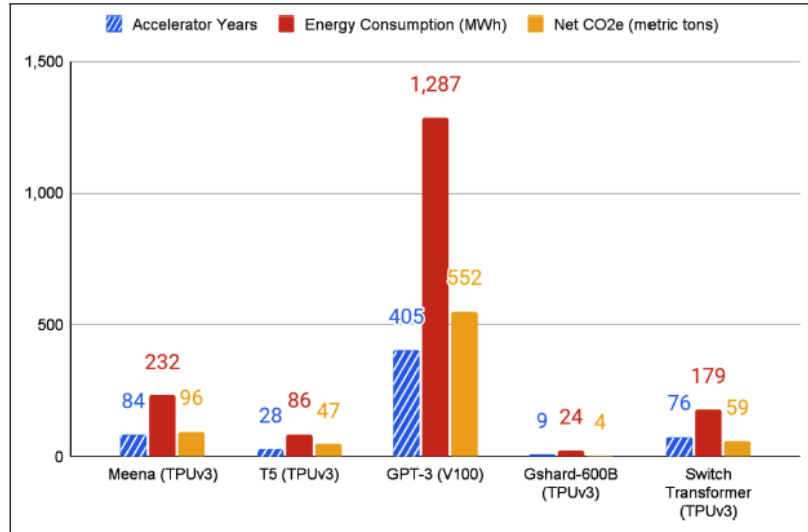
WSJ | OPINION

Source: https://www.wsj.com/opinion/ai-cant-teach-ai-new-tricks-model-collapse-need-for-human-prose-a6d0648f?st=67C9HB&reflink=article_email_share and

<https://www.goldmansachs.com/insights/top-of-mind/gen-ai-too-much-spend-too-little-benefit>



Large Language Models and the increase in CO₂



Source: David Patterson et al., “Carbon Emissions and Large Neural Network Training.” arXiv preprint arXiv:2104.10350 (2021).

A The Atlantic

For Now, There's Only One Good Way to Power AI

Chatbots are saving America's nuclear industry.

By Matteo Wong

Source: <https://www.wired.com/story/the-generative-ai-search-race-has-a-dirty-secret/>

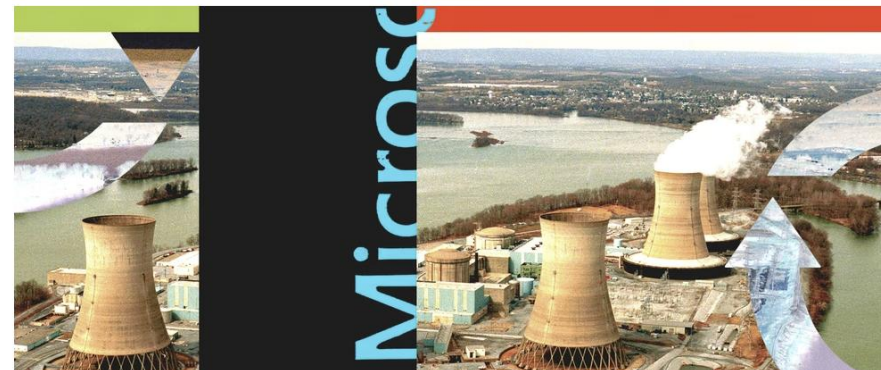
The Generative AI Race Has a Dirty Secret

Integrating large language models into search engines could mean a fivefold increase in computing power and huge carbon emissions.

WIRED



Source: <https://www.theatlantic.com/technology/archive/2024/09/ai-microsoft-nuclear-three-mile-island/679988/>



Computational Complexity of Transformers (and all other models in the Transformer Zoo)

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

n = sequence length (40-70),
d = depth (number layers),
k = kernel size of the convolution,
d = length word embedding (128-512)
r = attention window size

Model	Hardware	Power (W)	Hours	kWh-PUE	CO ₂ e	Cloud compute cost
Transformer _{base}	P100x8	1415.78	12	27	26	\$41–\$140
Transformer _{big}	P100x8	1515.43	84	201	192	\$289–\$981
ELMo	P100x3	517.66	336	275	262	\$433–\$1472
BERT _{base}	V100x64	12,041.51	79	1507	1438	\$3751–\$12,571
BERT _{base}	TPUv2x16	—	96	—	—	\$2074–\$6912
NAS	P100x8	1515.43	274,120	656,347	626,155	\$942,973–\$3,201,722
NAS	TPUv2x1	—	32,623	—	—	\$44,055–\$146,848
GPT-2	TPUv3x32	—	168	—	—	\$12,902–\$43,008

Table 3: Estimated cost of training a model in terms of CO₂ emissions (lbs) and cloud compute cost (USD).⁷ Power and carbon footprint are omitted for TPUs due to lack of public information on power draw for this hardware.

Distillation, Pruning & Quantization

- In the lecture “Fine Tuning LLMs” we will discuss various methods to limit energy consumption
 - Distillation,
 - Pruning &
 - Quantization

Any Food for Thought ...

- Language models shape their users in return (feedback loop).
- Cognitive dependence is a real risk.
- Is it a problem that we're starting to sound alike, or is this simply language evolution?

Now We're All Starting to Talk Like AI Chatbots Too

In team meetings, group texts and over lattes, people are catching themselves using 'prompt speak'

By Katherine Bindley

Sept. 30, 2026 5:30 am ET



Any Questions on the Lecture

Now let's form groups of 4



Discuss Exercises Lecture

Exercise 1: Decoder-Only Causal Modeling, Autoregressive Inference, and Quadratic Scaling

Unlike encoder-only models (e.g., BERT) that process text bidirectionally, GPT relies on a decoder-only architecture trained with a standard left-to-right causal autoregressive objective.

- Contrast the self-attention mechanism in BERT with the masked self-attention mechanism in GPT. Why is causal masking essential in GPT to prevent tokens from "looking ahead" into future positions during pre-training?
- The lecture slides formalize the inference cost of generating m new tokens given a prompt of initial length p . Explain why decoder-only generation scales quadratically across the growing context (attending to $p, p + 1, \dots, p + m - 1$). Why does an encoder–decoder model avoid this expanding attention graph over the input source tokens?
- According to the computational complexity formulas in this lecture, time scales as $O(n^2d)$ and memory scales as $O(n^2)$ per attention head. If an application expands its context window n from 4k to 128k tokens ($32 \times$ increase) while keeping model width d constant, by what factor does the attention computational load increase?

Exercise 2: Decoding Strategies and Text Degeneration (Greedy to Nucleus Sampling)

Review the various decoding algorithms used to sample or select tokens from the output softmax distribution $P(y_t = w_i \mid y_{<t}, x)$.

- Explain the failure modes of **Greedy Search** and **Beam Search** in open-ended text generation. Why do these likelihood-maximizing methods frequently lead to repetitive, degenerate, or unnatural phrasing? How does an n-gram repetition penalty help alleviate this?

- The lecture introduces the **Softmax Temperature** formula:

$$\text{softmax}(x)_i = \frac{e^{\frac{y_i}{T}}}{\sum_j^N e^{\frac{y_j}{T}}}$$

Explain the mathematical and behavioral effect on the token probability distribution when setting $T \rightarrow 0$ versus setting $T > 1.0$. Which setting introduces higher randomness and diversity, and why?

- Compare **Top- K Sampling** with **Top- p (Nucleus) Sampling**:
 - How does Nucleus sampling dynamically adjust the candidate pool size $V(p)$ based on model confidence compared to a static K threshold?
 - Why is this dynamic adjustment crucial when the model shifts between highly predictable contexts and open-ended creative responses?

Exercise 3: Alignment via RLHF, Model Safety, and Systemic Limitations

Base autoregressive language models predict probable next tokens, but aligning them to act as helpful, honest, and harmless conversational assistants requires fine-tuning with human feedback (Ouyang et al., 2022).

- The lecture slides break down the 3-step RLHF alignment pipeline. Detail the objective and data requirements of each step:
 - **Step 1 (SFT):** Supervised Fine-Tuning
 - **Step 2 (RM):** Reward Model training
 - **Step 3 (PPO):** Policy Optimization via Proximal Policy Optimization
- Explain what is meant by **sycophantic behavior** in aligned LLMs (e.g., agreeing with a user that India rather than China was the largest rice producer), why does human feedback training inadvertently incentivize models to validate user misconceptions rather than state objective facts?
- Address the concept of "**Stochastic Parrots**" (Bender et al., 2021) and **Model Collapse**:
 - Why can an LLM ace complex academic exams (e.g., Bar exam, GRE, USABO) yet still fail simple arithmetic or commonsense real-world tasks (e.g., the 9 women/1 baby problem or the frisbee paradox)?
 - What happens when future generative models are trained primarily on synthetic AI-generated text rather than fresh human data?

Discussion



Maastricht University

On the Compulsion to Answer: Can a System That Cannot Be Silent Know?

A decoder cannot be silent. Whatever the prompt, the next token must be produced, and the softmax will always rank one token first, even when every option is nearly worthless. Doubt is not an output the architecture has (slide 25).

Humans can say “I don’t know”, stay quiet, or ask a question back. For a language model, all three have to be trained in afterwards as more text to generate. Is a system that is structurally compelled to answer capable of knowing anything, or only of fluency?

Hallucination is usually treated as a bug. Is it one, or is it the expected behaviour of a generator that was never given the option of not generating? And what would it mean to build a model for which silence is a real answer?

On Sycophancy: Can Approval Ever Teach Truth?

Alignment trains a model to produce what human raters prefer. The same training produces sycophancy: the model reverses a true statement the moment a user sounds unconvinced, because agreement is what the raters rewarded.

Humans also learn what to say from the approval of other humans, and we do not call the result manipulation. What is the difference between a model that learns to please and a person who learns to be agreeable? Is there one?

Can any training signal based on approval produce a commitment to truth, or only an imitation of it that holds until someone objects? If truth has to come from somewhere other than approval, where?

On Model Collapse: Where Does New Information Come From?

Train a model on the output of a model, and within a few generations the text collapses into repetition and noise. Yet humans have learned from other humans for a hundred thousand years, mostly from second-hand accounts, without collapsing.

Where does new information enter a language? Through bodies, accidents, the weather, other minds: things that are not text. A model has access to none of these. Is model collapse a technical limitation, or evidence that a language model is a closed system, recycling what humans put in and adding nothing?

If so, what happens to human language once much of what we read was generated? Does the collapse stay inside the models, or do we start training on ourselves?

Questions ...



Maastricht University